



CC1002 - Introducción a la Programación

CLASE 6: RECURSIVIDAD PARTE 2

Departamento de Ciencias de la Computación (DCC) - FCFM
UNIVERSIDAD DE CHILE



Nadia Decar (nadiadecar@ug.uchile.cl)

Eduardo Godoy (egodoy@dcc.uchile.cl)

Series

- Determine la suma de $1 + 2 + 3 + \dots + n$

- Qué valor tiene la serie:

- Indicación

- Use la función potencia2()

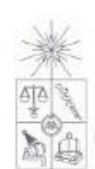
$$1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{16} + \frac{1}{32} + \dots + \frac{1}{2^n}$$

- Qué valor tiene la serie:

- Indicación

- Notar que es negativo si el número es par

$$1 - \frac{1}{2} + \frac{1}{3} - \frac{1}{4} + \frac{1}{5} - \frac{1}{6} + \dots - \frac{1}{n}$$



Suma1n = 1 + 2 + 3 + ... + n

```
# suma1n: int -> int
# dado un número n, determina el valor
# de la serie 1+2+3+ . . . + n
# Ejemplo: suma1n(1) retorna 1
#           suma1n(5) retorna 15

def suma1n(n):
    if n == 1:
        return 1
    else:
        return n + suma1n(n - 1)

# Test
assert suma1n(1) == 1
assert suma1n(5) == 15
```

Caso base

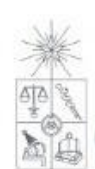
$n == 1 \rightarrow 1$
detiene la recursión

Caso recursivo

$n + \text{suma1n}(n - 1)$
reduce el problema en 1

Traza

$\text{suma1n}(4)$
 $= 4 + 3 + 2 + 1 = 10$



Series

- Determine la suma de $1 + 2 + 3 + \dots + n$

- Qué valor tiene la serie:

- Indicación

- Use la función potencia2()

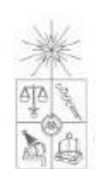
$$1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{16} + \frac{1}{32} + \dots + \frac{1}{2^n}$$

- Qué valor tiene la serie:

- Indicación

- Notar que es negativo si el número es par

$$1 - \frac{1}{2} + \frac{1}{3} - \frac{1}{4} + \frac{1}{5} - \frac{1}{6} + \dots - \frac{1}{n}$$



Unodiv2n =

$$1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{16} + \frac{1}{32} + \dots + \frac{1}{2^n}$$

```
# unodiv2n: int -> num
# dado un número n, determina el valor
# de la serie 1+1/2+1/4+1/8+...+1/(2^n)
# Ejemplo: unodiv2n(0) retorna 1
#           unodiv2n(5) retorna 1.96875

def unodiv2n(n):
    if n == 0:
        return 1
    else:
        return (1.0 / potencia2(n)) + unodiv2n(n - 1)

# Test
assert unodiv2n(0) == 1
assert unodiv2n(5) == 1.96875
```

Caso base

$n == 0 \rightarrow 1$
detiene la recursión

Caso recursivo

$1/2^n + \text{unodiv2n}(n-1)$
un término menos

Converge a 2

$\text{unodiv2n}(5)$
 $= 1.96875$



Series

- Determine la suma de $1 + 2 + 3 + \dots + n$

- Qué valor tiene la serie:

- Indicación

- Use la función potencia2()

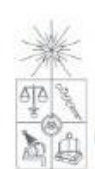
$$1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{16} + \frac{1}{32} + \dots + \frac{1}{2^n}$$

- Qué valor tiene la serie:

- Indicación

- Notar que es negativo si el número es par

$$1 - \frac{1}{2} + \frac{1}{3} - \frac{1}{4} + \frac{1}{5} - \frac{1}{6} + \dots + \frac{1}{n}$$



Masmenos1divn

$$1 - \frac{1}{2} + \frac{1}{3} - \frac{1}{4} + \frac{1}{5} - \frac{1}{6} + \dots - \frac{1}{n}$$

```
# masmenos1divn: int -> num
# dado un número n, determina el valor de
# la serie 1-1/2+1/3-1/4+ . . . 1/n
# Ejemplo: masmenos1divn(1) retorna 1
#           masmenos1divn(5) retorna 0.783333

def masmenos1divn(n):
    if n == 1:
        return 1
    else:
        if espar(n):
            return (-1.0/n) + masmenos1divn(n-1)
        else:
            return (1.0/n) + masmenos1divn(n-1)

assert masmenos1divn(1) == 1
assert masmenos1divn(2) == 0.5
```

Caso base

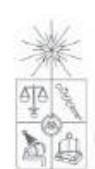
$n == 1 \rightarrow 1$
detiene la recursión

Signo alternado

espar(n) $\rightarrow -1/n$
impar $\rightarrow +1/n$

Ejemplo

masmenos1divn(5)
= 0.783333

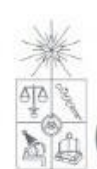


Ejercicio

- Programe la función $\cos(x, n)$ con la serie:

$$\cos(x, n) = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \dots + \frac{(-1)^n x^{2n}}{(2n)!}$$

- Use recursividad
- Use la función **factorial(n)** del apunte
- Implemente un diálogo simple para que el auxiliar pueda probar su ejercicio.
- Hint: x está en radianes.
- Propuesto: que funcione con grados.



Solución

```
def factorial(n):  
    if n == 0:  
        return 1  
    else:  
        return n * factorial(n - 1)  
  
def cos(x, n):  
    if n == 0:  
        return 1  
    else:  
        expresion = ((-1)**n * x**(2*n)) / factorial(2*n)  
        return expresion + cos(x, n - 1)
```

Serie de Chudnovsky: aproximar π

$$\pi = \frac{426880 \cdot \sqrt{10005}}{S(\infty)}$$

$$S(n) = \sum_{k=0}^n \frac{(-1)^k \cdot (6k)! \cdot (13591409 + 545140134 k)}{(3k)! \cdot (k!)^3 \cdot 640320^{3k}}$$

- Cada término aporta ~14 dígitos correctos: con $n = 2$ ya hay 40 decimales.
 - Recursión: **$S(0) = \text{término}(0)$** y **$S(n) = \text{término}(n) + S(n - 1)$**
-

Chudnovsky en Python (recursivo)

```
from math import factorial, sqrt

def termino(k):
    num = (-1)**k * factorial(6*k) * (13591409 + 545140134*k)
    den = factorial(3*k) * factorial(k)**3 * 640320**(3*k)
    return num / den

def suma(n):
    # S(n) recursivo
    if n == 0:
        return termino(0)
    return termino(n) + suma(n - 1)

def pi(n):
    return 426880 * sqrt(10005) / suma(n)

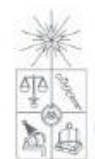
print(pi(2))    # 3.141592653589793
```

$$(-1)^k \cdot (6k)! \cdot (13591409 + 545140134 k)$$

$$(3k)! \cdot (k!)^3 \cdot 640320^{3k}$$

Turtle

Turtle

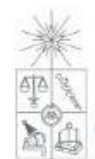


Turtle

- Idea original del lenguaje Logo
 - Desarrollado en 1996 por Wally Feurzig y Seymour Papert
- Esta "tortuga" parte ubicada en (0, 0)
 - Centro de la pantalla
- Puede caminar (forward / backward)
 - Si avanza, dibuja
 - Se puede levantar / bajar el lápiz (up, down)

Más información:

<https://docs.python.org/2/library/turtle.html>

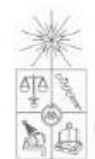


Turtle

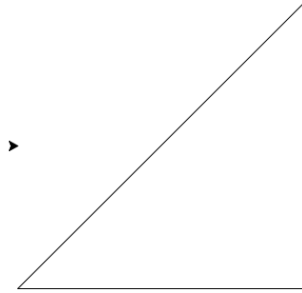
- Puede rotar:
 - `turtle.left(angulo)`
 - `turtle.rigth(angulo)`
 - angulo en grados, pero se puede cambiar:
 - `turtle.degrees()`, `turtle.radians()`
- Puede ser llevada a un punto (x, y):
 - `turtle.setposition(x, y)`
- Puede indicar dónde está
 - `x = turtle.xcor()`
 - `Y = turtle.ycor()`

Más información:

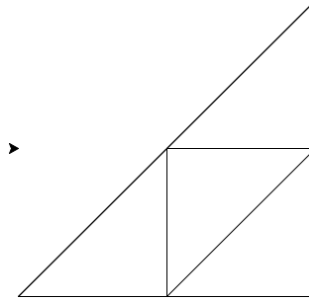
<https://docs.python.org/2/library/turtle.html>



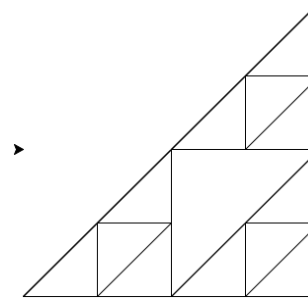
Triángulo de Sierpinski



$n = 1$

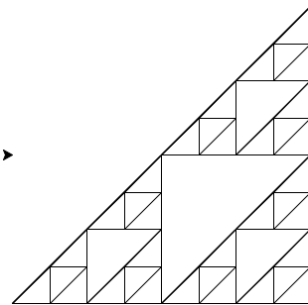


$n = 2$

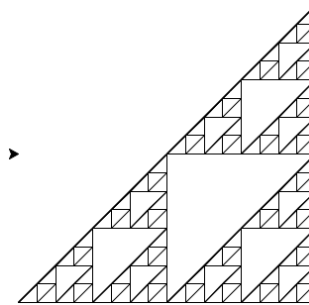


$n = 3$

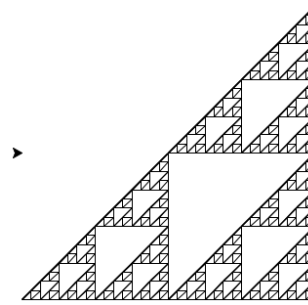
`dibujarSierpinski(0, 0, 300, 0, 300, 300, n)`



$n = 4$



$n = 5$



$n = 6$

Triángulo de Sierpinski

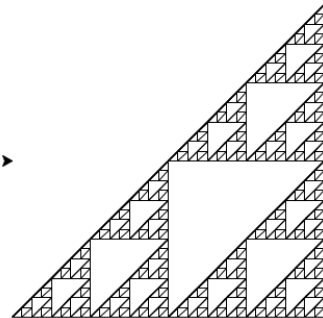
- Dibuja recursivamente el triángulo de Sierpinski en profundidad n:

dibujarSierpinski: num num num num num num int -->

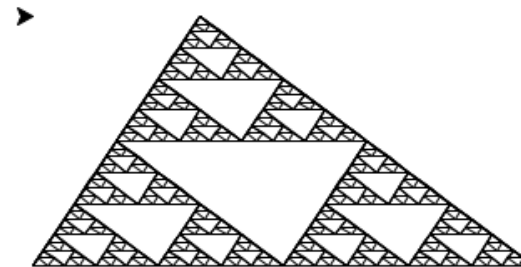
(xi, yi) son los vértices del triángulo

n es la profundidad, con $n \geq 1$

def **dibujarSierpinski**(x1, y1, x2, y2, x3, y3, n):

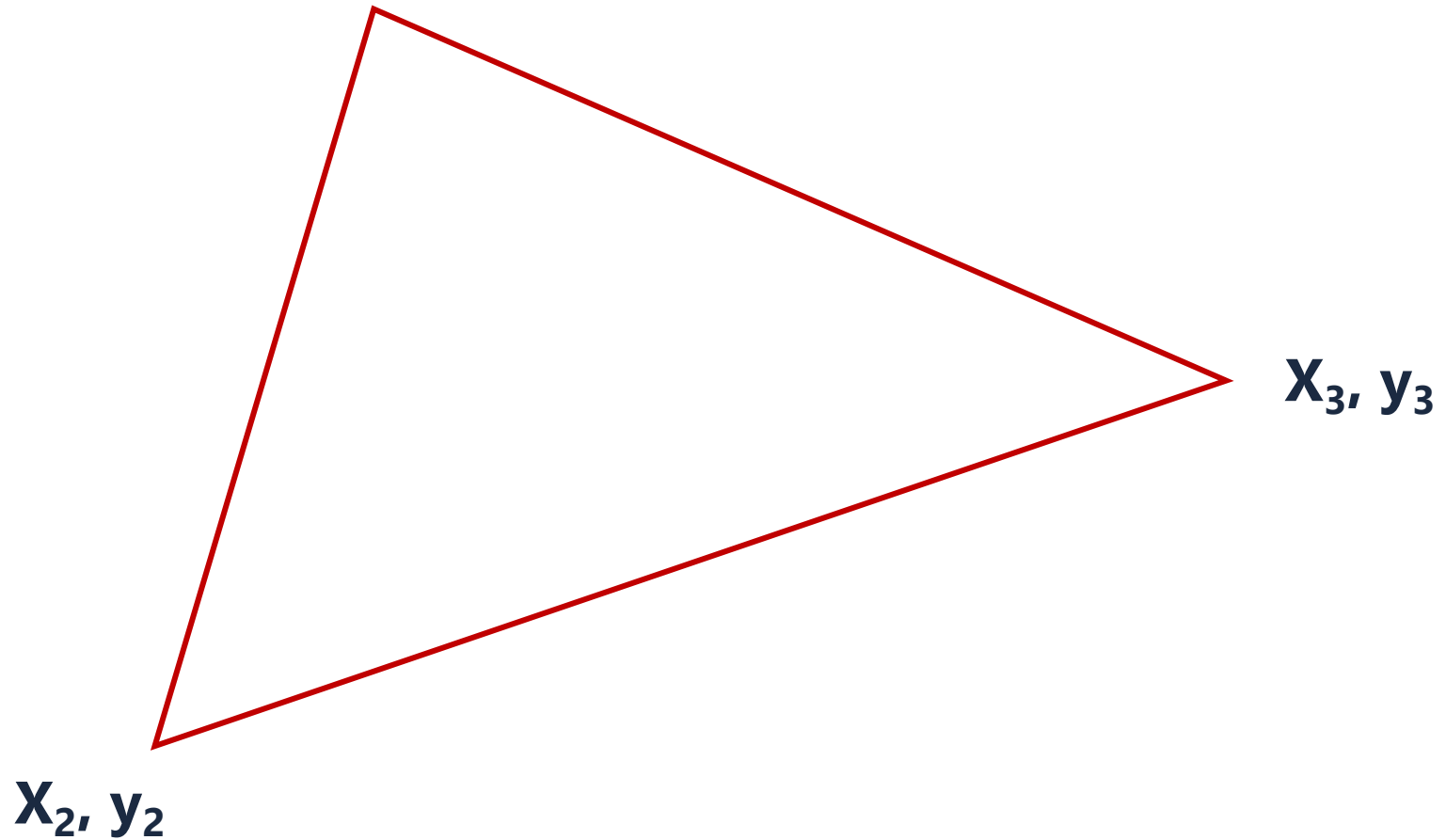


dibujarSierpinski(0, 0, 300, 0, 300, 300, 6)

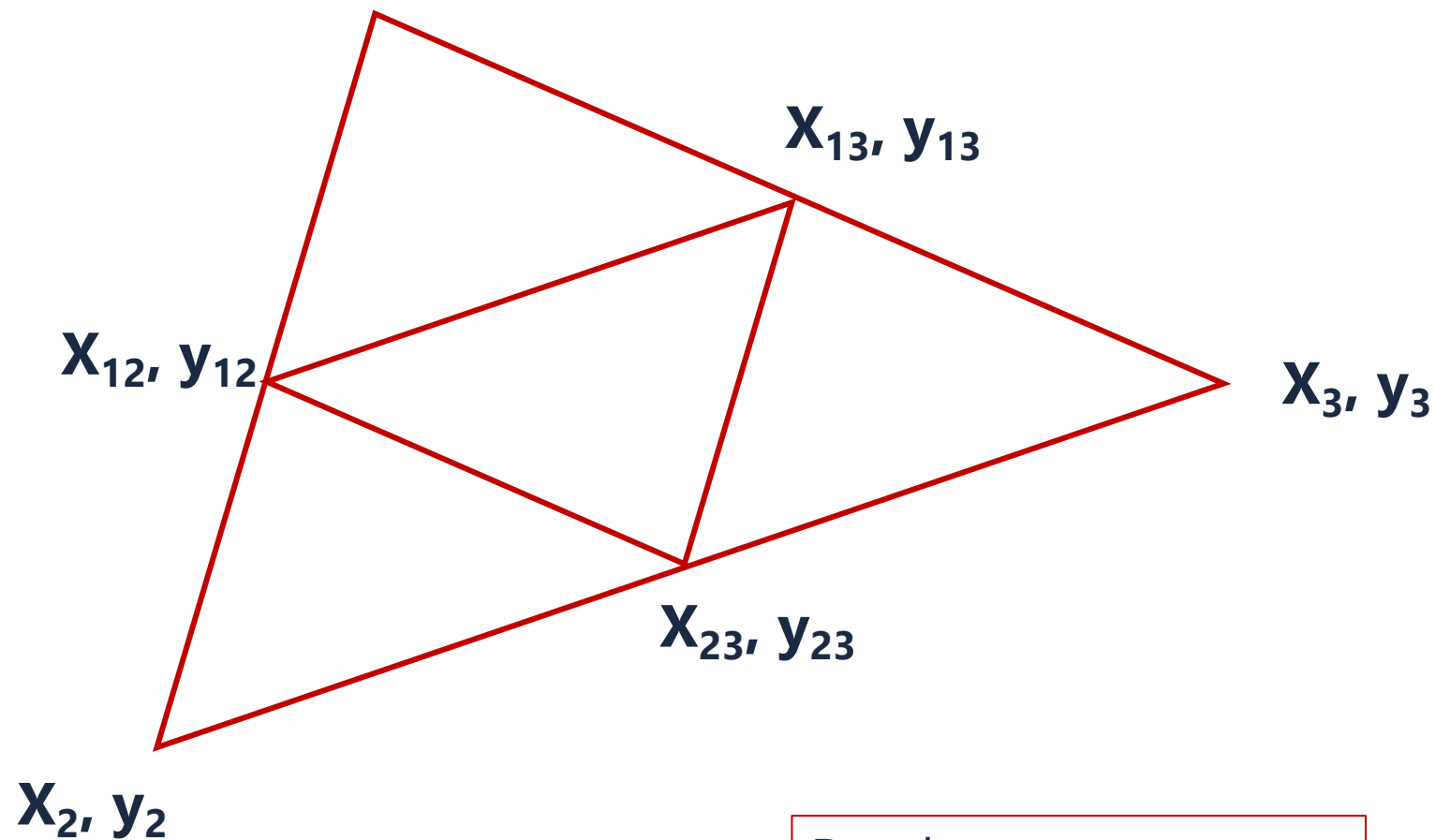


dibujarSierpinski(0, 0, 300, 0, 100, 150, 6)

Triángulo de Sierpinski



Triángulo de Sierpinski

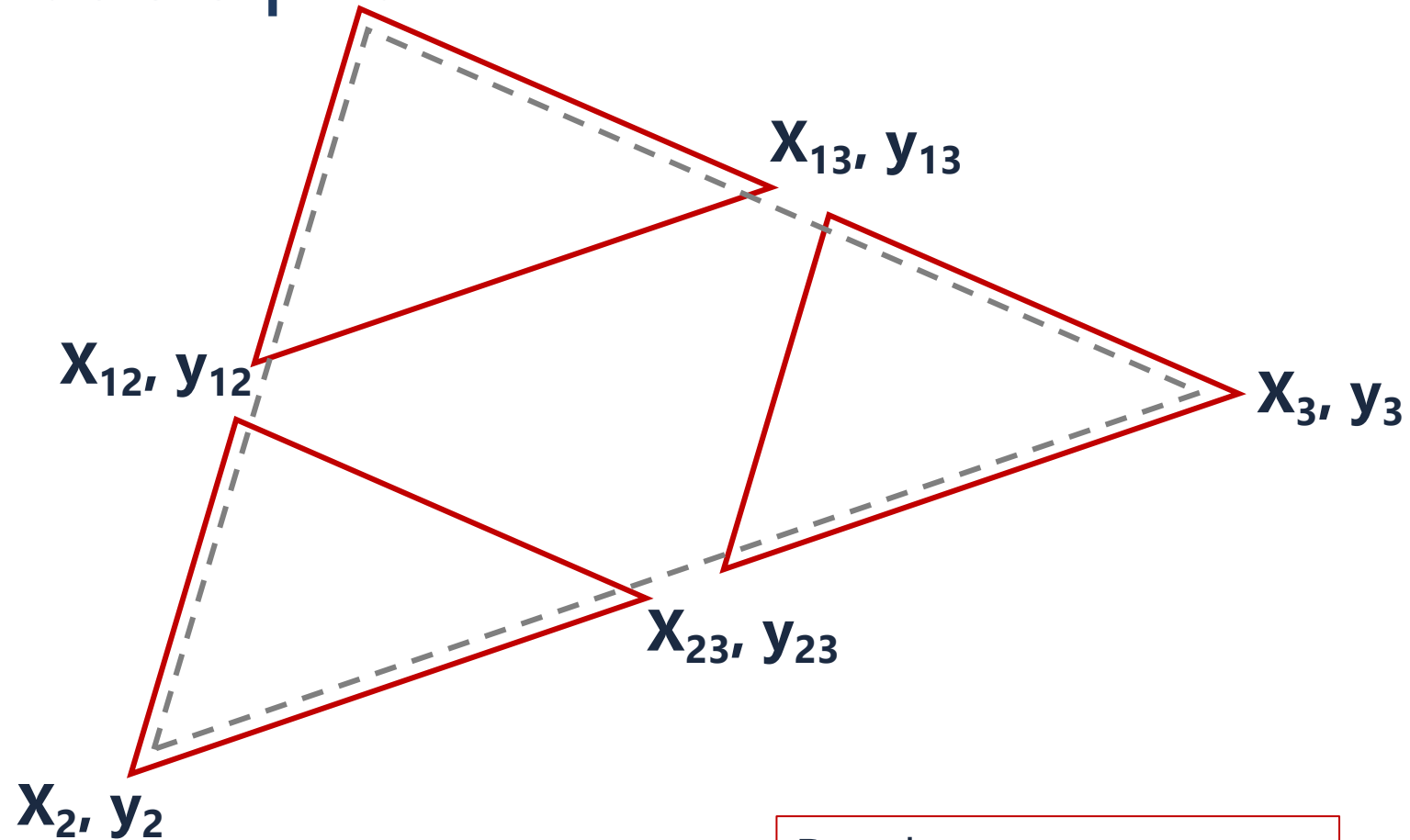


Donde:

$$x_{ab} = (x_a + x_b) / 2$$

$$y_{ab} = (y_a + y_b) / 2$$

Triángulo de Sierpinski



Donde:

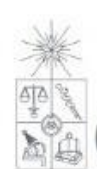
$$x_{ab} = (x_a + x_b) / 2$$

$$y_{ab} = (y_a + y_b) / 2$$

Dibujando una línea

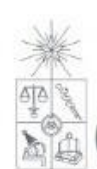
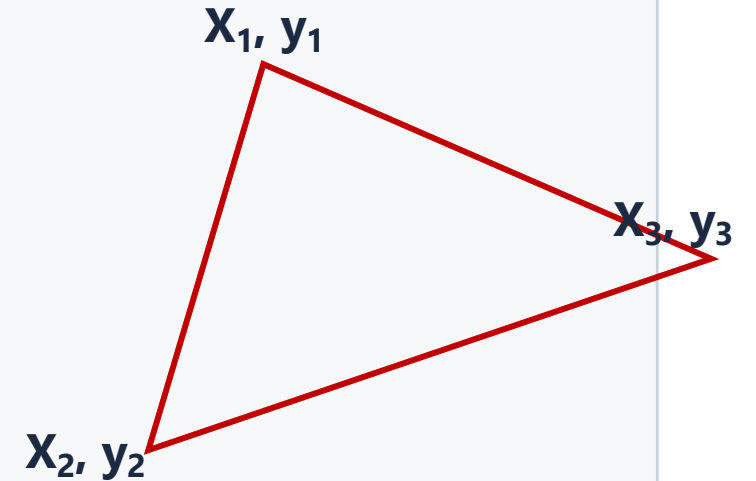
```
import turtle

# dibujarLinea: int int int int ->
# dado los puntos (x1, y1) y (x2, y2), dibuja una
# línea recta que une ambos puntos
# Ejemplo: dibujarLinea(0, 0, 200, 200)
def dibujarLinea(x1, y1, x2, y2):
    turtle.penup( )                # Se levanta el lápiz
    turtle.setposition(x1, y1)     # Se ubica el lápiz en (x1, y1)
    turtle.down( )                 # se baja el lápiz
    turtle.setposition(x2, y2)     # se avanza a la nueva posición, lo que dibuja
    la línea
```



Dibujando un triángulo

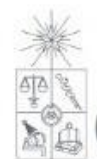
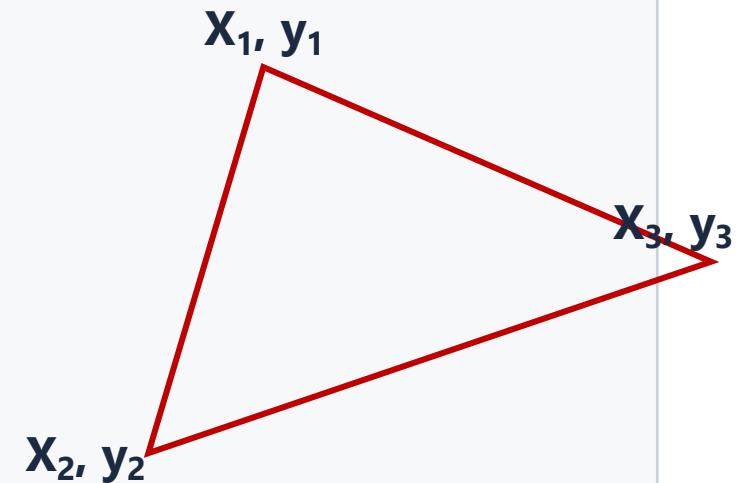
```
# dibujarTriangulo: int int int int int int -->  
# dada las coordenadas de tres puntos,  
# dibuja un triángulo  
def dibujarTriangulo(x1, y1, x2, y2, x3, y3):  
    dibujarLinea(x1, y1, x2, y2)  
    dibujarLinea(x2, y2, x3, y3)  
    dibujarLinea(x3, y3, x1, y1)
```



Dibujando un triángulo

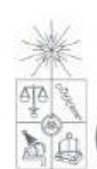
```
# dibujarTriangulo: int int int int int i  
# dada las coordenadas de tres puntos,  
# dibuja un triángulo  
def dibujarTriangulo(x1, y1, x2, y2, x3, y3):  
    dibujarLinea(x1, y1, x2, y2)  
    dibujarLinea(x2, y2, x3, y3)  
    dibujarLinea(x3, y3, x1, y1)
```

```
def dibujarLinea(x1, y1, x2, y2):  
    turtle.penup( )  
    turtle.setposition(x1, y1)  
    turtle.down( )  
    turtle.setposition(x2, y2)
```



Dado dos valores, retorna el valor medio

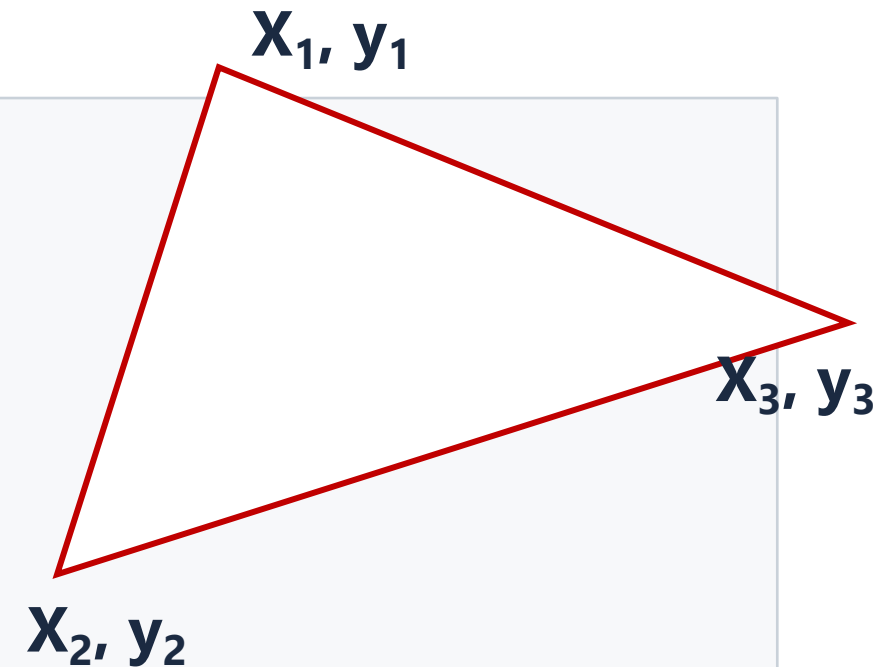
```
# medio: int int --> float
# dado dos valores, retorna el valor medio
# Ejemplo: medio(1, 2) retorna 1.5
def medio(a, b):
    return (a + b)/2.0
# test
assert cerca(medio(1, 2), 1.5, 0.0001)
```



```

# dibujarSierpinsky: int int int int int int int --> None
# dado un triángulo, lo dibuja y luego crea recursivamente
# tres nuevos triángulos formados por los puntos medios de
# cada lado del triángulo
def dibujarSierpinsky(x1, y1, x2, y2, x3, y3, n):
    if n >= 1:
        dibujarTriangulo(x1, y1, x2, y2, x3, y3)
        x12 = medio(x1, x2)
        y12 = medio(y1, y2)
        x23 = medio(x2, x3)
        y23 = medio(y2, y3)
        x13 = medio(x1, x3)
        y13 = medio(y1, y3)
        dibujarSierpinsky( x1, y1, x12, y12, x13, y13, n-1)
        dibujarSierpinsky(x12, y12, x2, y2, x23, y23, n-1)
        dibujarSierpinsky(x23, y23, x3, y3, x13, y13, n-1)

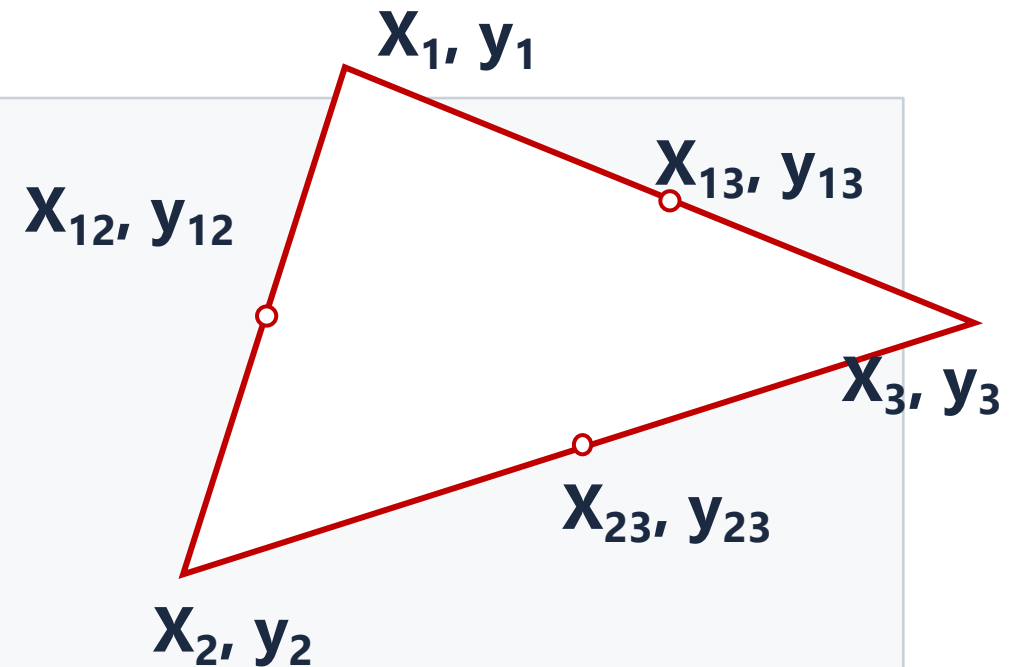
```



```

# dibujarSierpinsky: int int int int int int int --> None
# dado un triángulo, lo dibuja y luego crea recursivamente
# tres nuevos triángulos formados por los puntos medios de
# cada lado del triángulo
def dibujarSierpinsky(x1, y1, x2, y2, x3, y3, n):
    if n >= 1:
        dibujarTriangulo(x1, y1, x2, y2, x3, y3)
        x12 = medio(x1, x2)
        y12 = medio(y1, y2)
        x23 = medio(x2, x3)
        y23 = medio(y2, y3)
        x13 = medio(x1, x3)
        y13 = medio(y1, y3)
        dibujarSierpinsky(x1, y1, x12, y12, x13, y13, n-1)
        dibujarSierpinsky(x12, y12, x2, y2, x23, y23, n-1)
        dibujarSierpinsky(x23, y23, x3, y3, x13, y13, n-1)

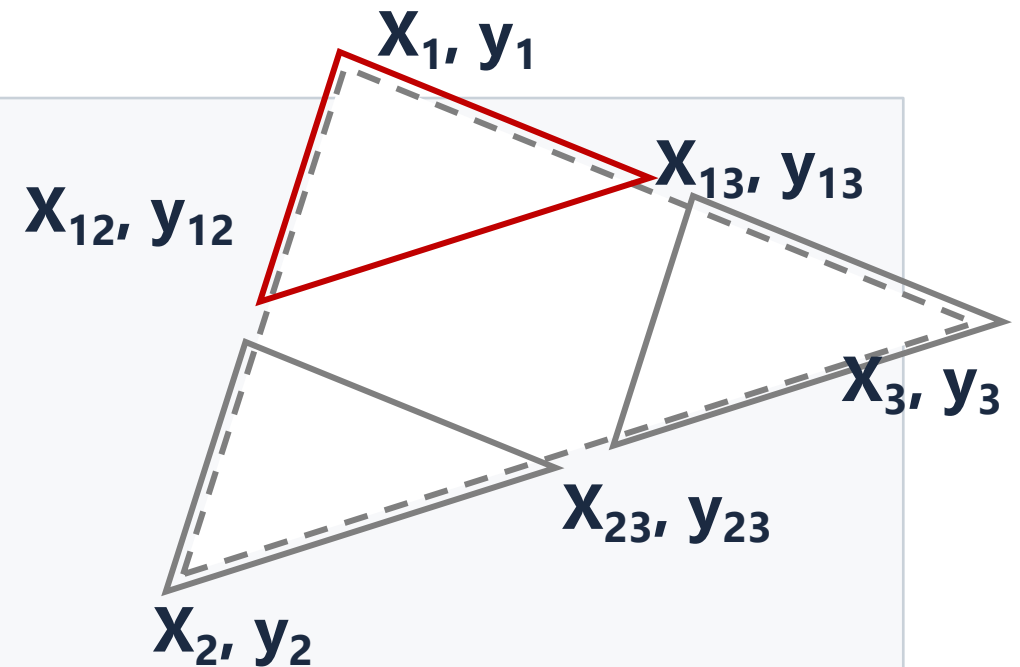
```



```

# dibujarSierpinsky: int int int int int int int --> None
# dado un triángulo, lo dibuja y luego crea recursivamente
# tres nuevos triángulos formados por los puntos medios de
# cada lado del triángulo
def dibujarSierpinsky(x1, y1, x2, y2, x3, y3, n):
    if n >= 1:
        dibujarTriangulo(x1, y1, x2, y2, x3, y3)
        x12 = medio(x1, x2)
        y12 = medio(y1, y2)
        x23 = medio(x2, x3)
        y23 = medio(y2, y3)
        x13 = medio(x1, x3)
        y13 = medio(y1, y3)
        dibujarSierpinsky( x1, y1, x12, y12, x13, y13, n-1)
        dibujarSierpinsky(x12, y12, x2, y2, x23, y23, n-1)
        dibujarSierpinsky(x23, y23, x3, y3, x13, y13, n-1)

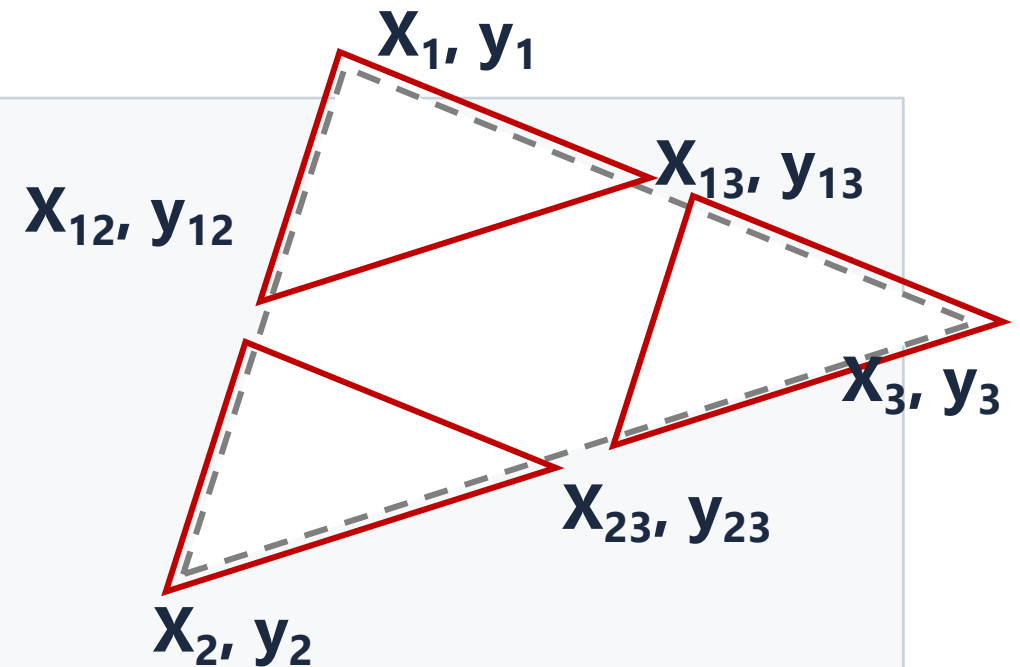
```



```

# dibujarSierpinsky: int int int int int int int --> None
# dado un triángulo, lo dibuja y luego crea recursivamente
# tres nuevos triángulos formados por los puntos medios de
# cada lado del triángulo
def dibujarSierpinsky(x1, y1, x2, y2, x3, y3, n):
    if n >= 1:
        dibujarTriangulo(x1, y1, x2, y2, x3, y3)
        x12 = medio(x1, x2)
        y12 = medio(y1, y2)
        x23 = medio(x2, x3)
        y23 = medio(y2, y3)
        x13 = medio(x1, x3)
        y13 = medio(y1, y3)
        dibujarSierpinsky( x1,   y1, x12, y12, x13, y13, n-1)
        dibujarSierpinsky(x12, y12,  x2,   y2, x23, y23, n-1)
        dibujarSierpinsky(x23, y23,  x3,   y3, x13, y13, n-1)

```



Propuesto

- Dado un número, escriba en pantalla los dígitos cuyo valor coincide con su posición, numerando las posiciones de derecha a izquierda.

- Ejemplo:

escribirEnPosicion(74123)

2

4



escribirEnPosicion(654103)

4

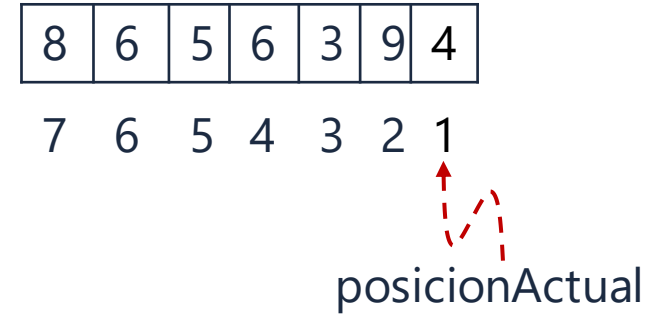
5



6

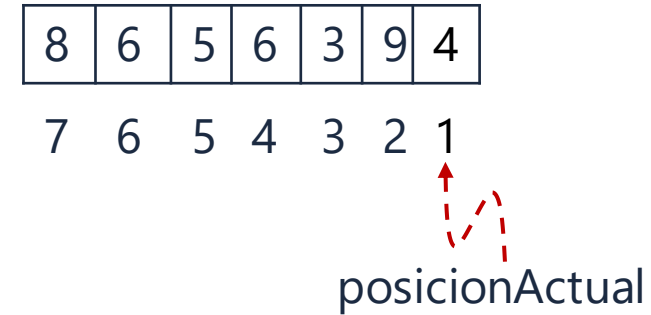
Solución del Propuesto

- El primer dígito es "simple"
 $\text{digito} = \text{numero} \% 10$
 $\text{posicionActual} = 1$
 if $\text{digito} == \text{posicionActual}$:
 print num

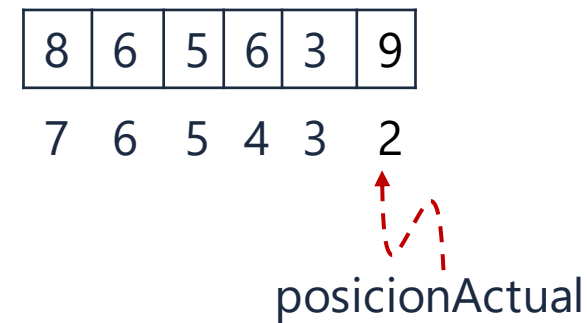


Solución del Propuesto

- El primer dígito es "simple"
 `digito = numero % 10`
 `posicionActual = 1`
 if `digito == posicionActual` :
 `print num`

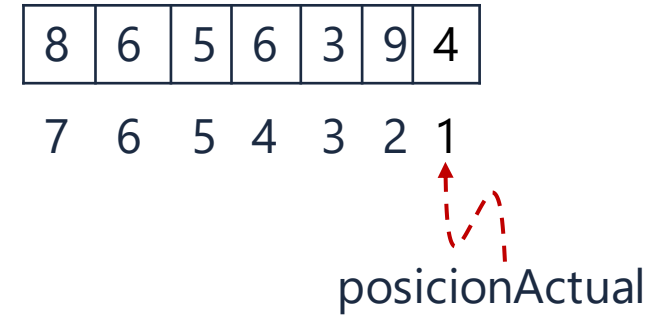


- Para el resto hay que recuperar la posición y el siguiente dígito
 `::`
 else:
 `::`
 if `digito == posicionActual`:
 `print nun`

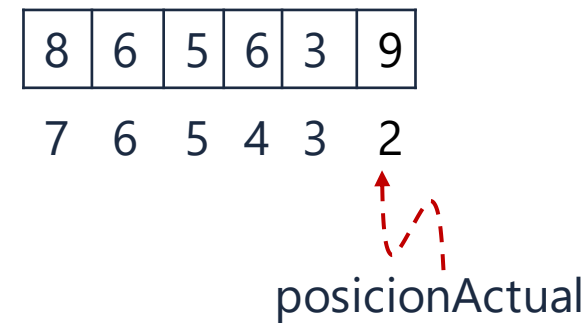


Solución del Propuesto

- El primer dígito es "simple"
 `digito = numero % 10`
 `posicionActual = 1`
 if `digito == posicionActual` :
 `print num`

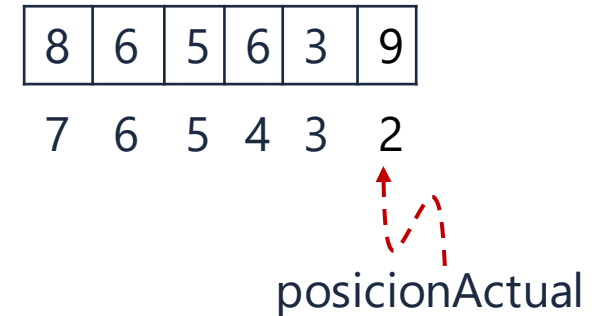
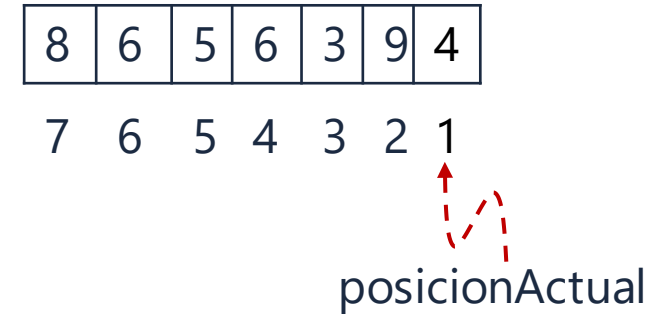


- Para el resto hay que recuperar la posición y el siguiente dígito
 `::`
 else:
 `::`
 if `digito == posicionActual`:
 `print nun`



Solución del Propuesto

- Dos variables que cambian
 $\text{digito} = \text{numero} \% 10$
 $\text{posicionActual} = \text{posicionActual} + 1$
- Pero la función solo tiene 1 parámetro:
 def escribirEnPosicion(numero):
 ::
- Solución:
 - Crear una función auxiliar con más parámetros



```
def escribirEnPosicion(numero):
```

```
    :  
    :  
    :
```

```
    recorrerNumero(numero, posicionActual)
```

← Función con un parámetro

Llama a

← Función con dos parámetros

```
def recorrerNumero(num, posActual):
```

```
    if num < 10:
```

```
        if (num == posActual):  
            print(num)
```

```
        return
```

```
    else:
```

```
        digito = num % 10
```

```
        if digito == posActual:  
            print(digito)
```

```
        recorrerNumero(num // 10, posActual + 1)
```

```
def escribirEnPosicion(numero):  
    posicionActual = 1  
    recorrerNumero(numero, posicionActual)
```

```
def recorrerNumero(num, posActual):  
    if num < 10:  
        if (num == posActual):  
            print(num)  
        return  
    else:  
        digito = num % 10  
        if digito == posActual:  
            print(digito)  
  
        recorrerNumero(num // 10, posActual + 1)
```

8	6	5	6	3	9	4
---	---	---	---	---	---	---

7 6 5 4 3 2 1



posicionActual

8	6	5	6	3	9
---	---	---	---	---	---

7 6 5 4 3 2



posicionActual

Ejercicio

Escriba una función que determine si un número es palíndromo.

Ejemplos

`esPalindromo(3456)` → `False`

`esPalindromo(3443)` → `True`

Hint

Escriba una función recursiva que invierta un número natural:

`invertirNatural(3456)` → `6543`

Solución: invertirNatural y esPalindromo

invertirNatural

```
# invertirNatural: int -> int
# invierte los dígitos de un natural
# Ejemplo: invertirNatural(1234)
#           retorna 4321

def invertirNatural(n, inv = 0):
    if n == 0:
        return inv
    else:
        return invertirNatural(n // 10,
                                inv * 10 + n % 10)
```

esPalindromo

```
# esPalindromo: int -> bool
# True si el número se lee igual
# de izquierda a derecha y al revés
# Ejemplo: esPalindromo(12321) -> True

def esPalindromo(n):
    return n == invertirNatural(n)

# Test
assert invertirNatural(1234) == 4321
assert esPalindromo(12321)
assert not esPalindromo(1231)
```

Opcion 2 para invertirNatural

```
def cantidadDigitos(n):  
    if n < 10:  
        return 1  
    return 1 + cantidadDigitos(n // 10)  
  
def invertirNatural(n):  
    if n < 10:  
        return n  
    else:  
        return ((n % 10) * 10**(cantidadDigitos(n) - 1)  
                + invertirNatural(n // 10))  
  
# invertirNatural(3456) -> 6543
```

Idea

Sin acumulador: el último dígito se eleva a la posición más alta.

Requiere conocer cuántos dígitos tiene n.

Traza 3456

$$\begin{aligned} &6 * 10^3 + \text{inv}(345) \\ &\quad 5 * 10^2 + \text{inv}(34) \\ &\quad\quad 4 * 10 + 3 \\ &= 6543 \end{aligned}$$

Ejercicio

Escriba una función que elimine dígitos duplicados adyacentes.

Ejemplos

`eliminaDuplicados(3456)` → 3456

`eliminaDuplicados(3443)` → 343

Hint

Compare el último dígito con el penúltimo y recurra sobre `n // 10`:

`n % 10 == (n // 10) % 10 ?`
